# Exercise Sheet 3: Side Channels

Information Security Lab

October 30, 2026

Answer Key

## Exercise 1: The Sound of Insecurity

**Setting**

You are trying to open a safe protected by a combination lock with four dials. Each dial has digits from 0 to 9, and exactly one combination opens the safe.
You do not know the combination, but the lock imposes no limit on the number of attempts. You can only test complete combinations. For each attempt, you learn whether the safe opens.

**Task 1.** What is the maximum number of complete combinations you need to test to determine the correct combination?

**Solution**

Each of the four dials has 10 possible values, giving $10^4 = 10{,}000$ combinations.
After testing 9,999 incorrect combinations, the attacker can identify the remaining combination as correct.
Therefore, at most 9,999 tests are needed to determine the combination, followed by one final attempt to open the safe.

**Setting**

You notice that turning a dial produces a distinctive click when it reaches a certain digit. For example, the first dial clicks at 8 and the second at 5. You turn each dial until you hear a click, and the safe opens.

**Task 2.** What information does the clicking sound reveal? How could you use it to open another safe with the same lock model?

**Solution**

The clicking sound reveals whether the selected digit is correct.
An attacker can use this information to identify each digit individually instead of testing complete combinations.

**Task 3.** What is the maximum number of individual dial positions you need to test to determine the correct combination?

**Solution**

Each dial has 10 possible digits. After testing nine incorrect digits, the remaining digit must be correct.
An attacker therefore needs at most $4 \times 9 = 36$ tests to determine the combination.
The attacker can then enter the identified combination to open the safe.

**Task 4.** How could the manufacturer prevent or mitigate this attack?

**Solution**

The manufacturer could:

- Eliminate distinguishable clicking sounds.
- Limit the number of unsuccessful attempts.
- Verify only the complete combination rather than individual digits.

These measures reduce the information available to an attacker or limit opportunities to exploit it.

**Task 5.** Suppose the manufacturer introduces random clicking sounds while the dials are turning.

Does this prevent the attack?

**Solution**

Not necessarily. If the correct digit still produces a statistically distinguishable sound, an attacker may identify it through repeated measurements.
Adding noise makes the attack more difficult but does not necessarily eliminate the information leakage.

**Task 6.** How could a similar attack be used to extract information from a Trusted Execution Environment (TEE)?

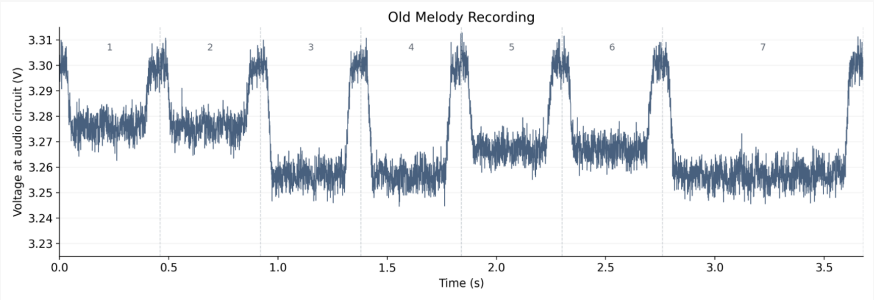
**Solution**

Computations inside a TEE may produce distinguishable sounds through hardware components, such as cooling fans or electronic components.
If these sounds depend on secret-dependent computations, an attacker could analyze acoustic measurements to infer information about the secrets without directly accessing protected memory.

## Exercise 2: The Power of Secrets

**Setting**

You find an old toy instrument that can record and replay melodies. Unfortunately, its speaker is broken. You connect an oscilloscope to the output of the instrument's audio circuit and record the voltage while replaying a previously recorded melody.
You obtain the following trace:




**Task 1.** What information could you infer from the voltage trace? How could you test your hypothesis?

**Solution**

The distinct voltage levels may correspond to different notes, while the duration of each interval may indicate how long a note was played.
An attacker could test this by playing known notes individually and comparing their voltage traces.

**Setting**

You measure the average output voltage while playing each note individually and obtain the following values:

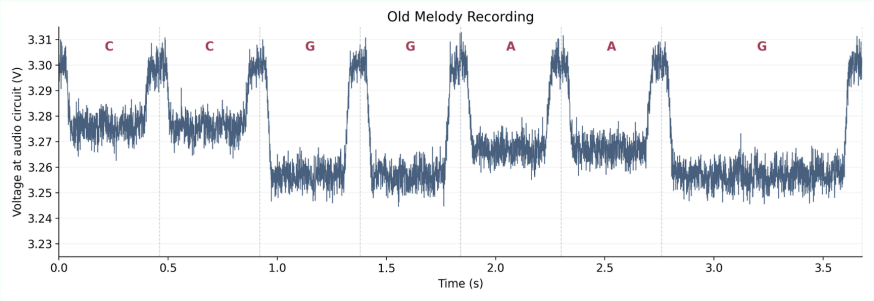
| Note | Average Voltage (V) |
|---|---|
| C | 3.275 |
| D | 3.240 |
| E | 3.295 |
| F | 3.300 |
| G | 3.255 |
| A | 3.265 |
| B | 3.235 |

Assume that each note produces a distinguishable and stable average voltage.

**Task 2.** Can you recover the recorded melody from the voltage trace?

**Solution**

Yes. By comparing the voltage levels in the trace with the measured values, an attacker can identify the notes and their approximate durations.
The recovered melody is:




**Task 3.** How could a similar attack be used to extract information from a TEE?

**Solution**

An attacker could monitor power consumption while the enclave processes secret data.
If power consumption depends on the secret-dependent computation, the attacker may infer information about the secret without directly accessing enclave memory.
Check out PLATYPUS (IEEE S&P 2021).

## Exercise 3: The Prime Suspect

**Setting**

You control the OS of a system running an Intel SGX enclave. The enclave processes a secret number $n$ that you cannot read directly.

The enclave first performs preliminary computations taking 100 ms. It then checks whether $n$ is prime by testing divisors in ascending order, starting at 2. The enclave stops at the first divisor of $n$, or after checking all integers up to $\lfloor\sqrt{n}\rfloor$. Each divisibility test takes approximately 2 ms.
Although you cannot inspect the enclave's memory, you can measure its total execution time. Assume that scheduling overhead, interrupts, and other sources of timing variation are negligible.
You observe that the enclave takes 108 ms to process the secret number.

**Task 1.** Assuming $n \in \{21, 24, 29, 33\}$, can you determine the secret number from the observed execution time?

**Solution**

The preliminary computations take 100 ms, leaving 8 ms for divisibility tests. At 2 ms per test, the enclave performed approximately four tests.
The expected execution times are:

| Candidate | Divisors Tested | Time (ms) |
|---|---|---|
| 21 | 2, 3 | 104 |
| 24 | 2 | 102 |
| 29 | 2, 3, 4, 5 | 108 |
| 33 | 2, 3 | 104 |

The attacker can identify 29 as the secret number from the observed execution time, assuming the timing model is sufficiently accurate.

**Task 2.** Suppose the enclave instead takes 104 ms. Can you determine the secret number?

**Solution**

An execution time of 104 ms corresponds to approximately two divisibility tests.
Both 21 and 33 require two tests, so the attacker cannot distinguish between them using execution time alone.

**Task 3.** Does SGX prevent this timing side-channel attack? Explain your answer.

**Solution**

No. SGX protects the confidentiality and integrity of enclave memory but does not hide execution time. A malicious OS can measure how long an enclave executes without accessing its protected memory. If execution time depends on secret data, the attacker may infer information about the secret.

## Exercise 4: It's Not My Fault!

**Setting**

You are analyzing an application running inside an Intel SGX enclave.

```
void common_work() {
    // Perform some computation
}

void option_A() {
    common_work();
}

void option_B() {
    common_work();
}

void option_C() {
    common_work();
}

void process_secret(int secret) {
    if (secret == 0)
        option_A();
    else if (secret == 1)
        option_B();
    else
        option_C();
}
```

The enclave processes a `secret` value that determines which of three functions is executed. As the attacker, you control the OS. Although you cannot directly read enclave memory, you can manipulate page permissions to induce and observe page faults.
Assume the following:

- Page size is 4 KiB.
- The enclave is loaded with a page-aligned load bias of `0x70000000`.
- Each function fits entirely within a single page.
- All function calls and returns occur as shown (no inlining, tail-call optimization, or other changes to the illustrated control flow).
- Assume an idealized observation model in which the OS induces an instruction page fault whenever execution enters a different code page, including when returning to a previously visited page.
- Consider only instruction page faults caused by the functions shown.
- Trace collection begins with the first instruction of `process_secret()` and ends after its final return.

**Task 1.** You mark an enclave code page as not present. When the enclave attempts to execute

an instruction on that page, a page fault occurs. You handle the fault and restore the page-table entry.

Will every subsequent instruction fetch from that page cause another page fault?

**Solution**

No. Once the mapping is restored, subsequent instruction fetches can proceed without additional page faults. The processor may also cache the address translation in the Translation Lookaside Buffer (TLB). To induce another fault, the attacker must modify the page mapping or permissions again and invalidate stale TLB translations when necessary.

**Task 2.** You inspect the enclave binary and find that the functions are located at the following offsets relative to the enclave's load address:

| Function | Offset |
|---|---|
| `process_secret()` | `0x1100` |
| `option_A()` | `0x2100` |
| `option_B()` | `0x2200` |
| `option_C()` | `0x2300` |
| `common_work()` | `0x3100` |

Construct the instruction page-fault trace for each possible value of `secret`.

**Solution**

The runtime address of each function is obtained by adding its offset to the enclave's load address.
All three option functions reside on the same code page and call `common_work()`.
Therefore, every secret value produces the same trace:

```
0x70001000 -> 0x70002000 -> 0x70003000
           -> 0x70002000 -> 0x70001000
```

**Task 3.** Can you distinguish the value of `secret` from the instruction page-fault trace?

**Solution**

No. All secret values produce the same trace, so the attacker cannot distinguish them using this information alone.

**Setting**

The developer modifies the enclave so that `option_C()` calls `special_work()` instead of `common_work()`.

```
void common_work() {
    // Perform some computation
}

void special_work() {
    // Perform some other computation
}

void option_A() {
    common_work();
}

void option_B() {
    common_work();
}

void option_C() {
    special_work();
}

void process_secret(int secret) {
    if (secret == 0)
        option_A();
    else if (secret == 1)
        option_B();
    else
        option_C();
}
```

**Task 4.** You inspect the modified enclave binary and find that the functions are located at the following offsets relative to the enclave's load address:

| Function | Offset |
|---|---|
| `process_secret()` | 0x1100 |
| `option_A()` | 0x2100 |
| `option_B()` | 0x2200 |
| `option_C()` | 0x2300 |
| `common_work()` | 0x3100 |
| `special_work()` | 0x4100 |

Construct the instruction page-fault trace for each possible value of `secret`.

**Solution**

For `secret` equal to 0 or 1:

```
0x70001000 -> 0x70002000 -> 0x70003000
           -> 0x70002000 -> 0x70001000
```

For all other secret values:

```
0x70001000 -> 0x70002000 -> 0x70004000
           -> 0x70002000 -> 0x70001000
```

**Task 5.** Can you distinguish the value of `secret` from the instruction page-fault trace?

**Solution**

Partially. The attacker can determine whether `secret` belongs to $\{0, 1\}$ or lies outside this set, but cannot distinguish 0 from 1.

**Setting**

The developer proposes enabling address space layout randomization (ASLR). Consider a hypothetical recompilation in which `option_A()`, `option_B()`, and `option_C()` occupy three distinct code pages, at offsets `0x2000`, `0x3000`, and `0x4000`, respectively. Instead of using a fixed load bias, the enclave is loaded at a randomly chosen page-aligned base address on each execution. Assume you do not initially know the load bias but can observe the instruction page-fault trace.

**Task 6.** Does randomizing the load bias prevent you from distinguishing the secret values?

**Solution**

No, provided the attacker can identify the function pages relative to the enclave's base address. ASLR changes absolute addresses but preserves relative page offsets. The attacker can therefore distinguish the option functions using their relative positions. Moreover, in the SGX threat model, the untrusted OS participates in memory management and can generally observe or influence the enclave's load address. Thus, ASLR alone does not prevent secret-dependent page-fault leakage.

**Task 7.** Consider the original implementation of `process_secret()`, where all three options call `common_work()`. Modify the implementation so that:

- Its behavior remains unchanged.
- The instruction page-fault trace is independent of `secret`.
- Unnecessary computation is minimized.

Provide a concrete implementation and explain why it prevents the attack.

**Solution**

Several approaches can prevent secret-dependent instruction page-fault traces.

**Defense A: Eliminate unnecessary branching.**
Since all options perform the same computation, `common_work()` can be called directly:

```
void process_secret(int secret) {
    common_work();
}
```

This removes unnecessary branches and function calls. Every secret value follows the same execution path, producing the same instruction page-fault trace.

**Defense B: Execute all options.**

Another approach is to execute every option:

```
void process_secret(int secret) {
    option_A();
    option_B();
    option_C();
}
```

The execution order is independent of the secret, so the instruction page-fault trace is identical for all secret values.
However, this introduces redundant computation and may change program behavior if the functions have side effects.

**Defense C: Use branchless selection.**
For computations that produce different results, branchless selection is another possible approach. For example:

```
uint32_t process_secret(uint32_t secret) {
    uint32_t a = 10, b = 20, c = 30;

    uint32_t mask_a = 0u - (secret == 0);
    uint32_t mask_b = 0u - (secret == 1);
    uint32_t mask_c = ~(mask_a | mask_b);

    return (a & mask_a)
         | (b & mask_b)
         | (c & mask_c);
}
```

Each mask is either all zeros or all ones, selecting the correct result without explicit secret-dependent branching.
This example illustrates branchless selection rather than a direct replacement for the original functions. The generated machine code must still be checked for secret-dependent control flow or memory accesses.

**Conclusion.**
Defense A is preferable for the original implementation because it preserves the computation while eliminating unnecessary work.
Preventing instruction page-fault leakage does not necessarily prevent other side channels, such as timing, cache accesses, data page faults, or power consumption.

**Task 8.** So far, we have assumed that the observed page-fault trace contains only accesses to the relevant enclave code pages. In practice, other code may execute and additional page faults may occur during trace collection.

How might this affect the observed trace, and what challenge does it introduce for the attacker?

**Solution**

The observed trace may contain additional page faults unrelated to the secret-dependent execution. This introduces noise, making it harder for the attacker to identify the relevant sequence of page accesses and infer the secret.
The attacker may need to filter irrelevant faults or collect multiple traces to distinguish meaningful patterns.

**Setting**

To mitigate the instruction page-fault side channel, the developer changes the implementation as follows:

```
uint8_t values[8192];

uint8_t process_secret(uint16_t secret) {
    return values[secret];
}
```

Assume the following:

- The array starts at the page-aligned address `0x70005000`.
- Each array element occupies one byte.
- The secret is an integer between 0 and 8191.
- The OS can induce and observe data page faults.
- Ignore all other memory accesses.

**Task 9.** Does the modified implementation prevent the attacker from learning information about `secret` through page faults? If not, explain what information the attacker can learn and whether the exact secret value can be determined.

**Solution**

No. Although the instruction execution path is independent of the secret, the memory address accessed depends on its value.
Since each page holds 4096 bytes, the array spans two pages:

- `0x70005000`: entries 0–4095.
- `0x70006000`: entries 4096–8191.

By observing the data page fault, the attacker can determine whether secret $< 4096$ or secret $\geq 4096$.
However, the attacker cannot determine the exact secret value because accesses within the same page are indistinguishable at page granularity. Thus, eliminating secret-dependent control flow does not necessarily prevent page-fault side channels. Secret-dependent data accesses can still leak information.